TRANSPORT AND TELECOMMUNICATION INSTITUTE

ENGINEERING FACULTY

Laboratory work N1

Course: **Programming 2**

Theme: **User-defined functions**

Student: Igors Oļeiņikovs

Student code: 93642

Group: 4501BTA

Riga
2026

CONTENTS

1. Laboratory work task 3
2. Individual task 3
3. Algorithm data flow 4
4. Algorithm description 7
5. Source code 8
6. Running program example 11
7. Testing 12
8. Conclusions 13

1. LABORATORY WORK TASK

Create an algorithm for the function according to the individual task. Create in C++ two overloaded functions in one source file: the first function must process one c-string passed by pointer, the second function must process one value of type string passed by reference. If returning a value is required, it may be returned through the return operator or through output parameters. Parameters that are not changed must be declared as const. In main, create a two-dimensional array of chars that stores c-strings entered by the user and create a second array that stores the same values as objects of type string. Call the appropriate function for every string in each array and print the result. All input and output must be implemented only in main. Print the individual task number in the beginning of the program, test the application, and prepare a report.

2. INDIVIDUAL TASK

Individual task number is calculated as 93642 % 25 = 17.

Task 17: Function returns the shortest word in the string. Words are considered to be separated by one space. If there are several shortest words, the function may return any of them.

3. ALGORITHM DATA FLOW

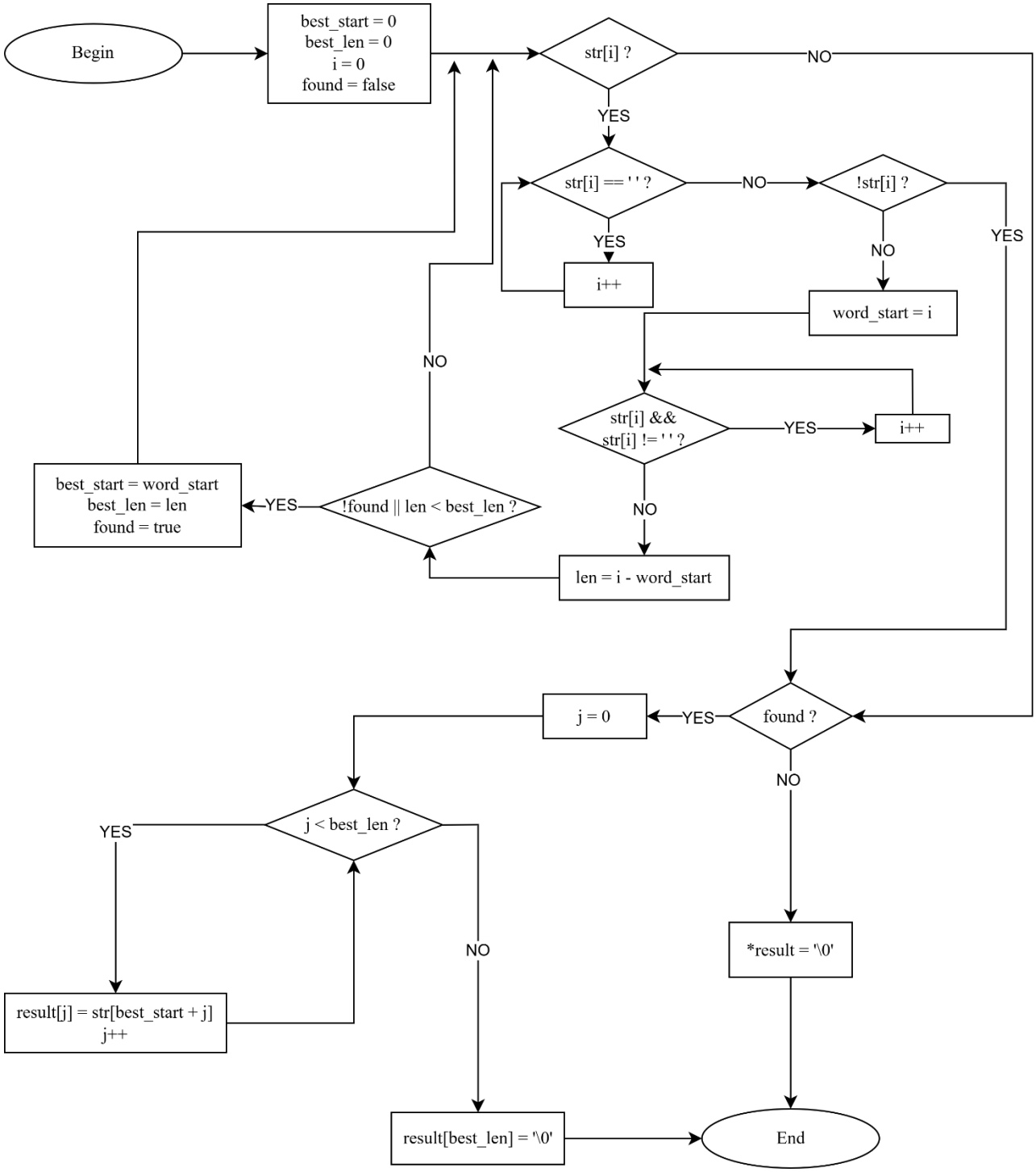

Fig. 1. **C-string overloaded function**

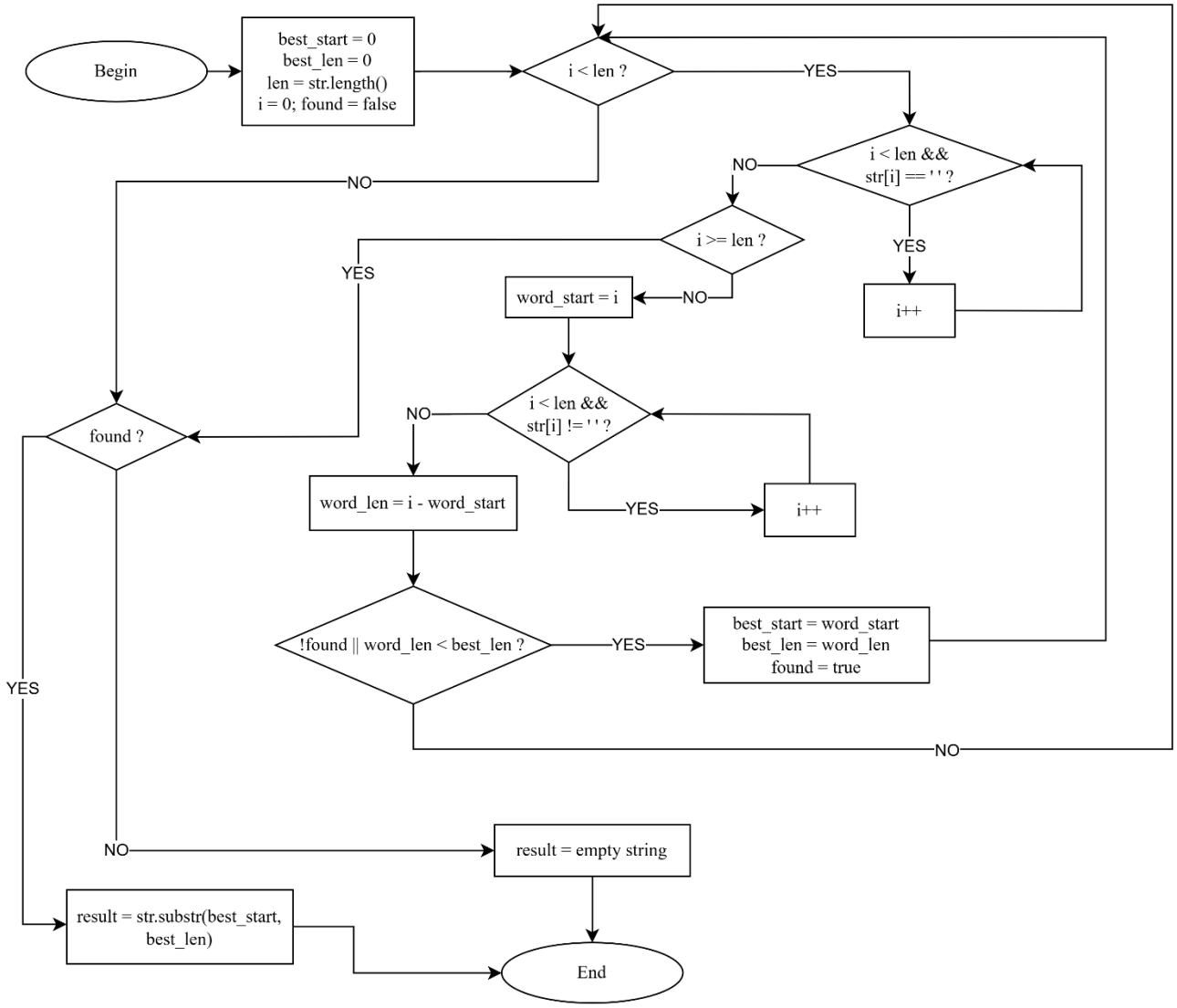

Fig. 2. **String object overloaded function**

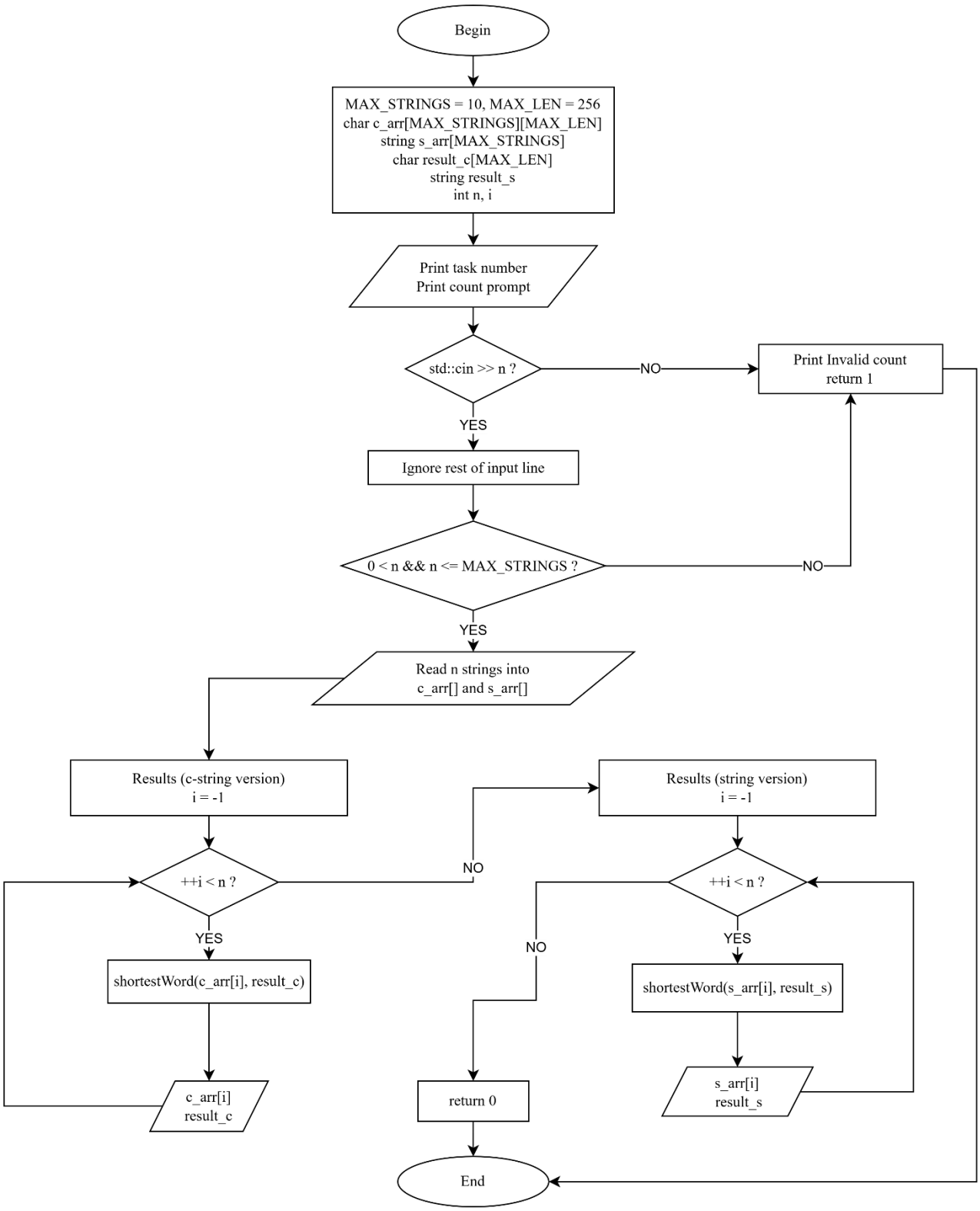

Fig. 3. **Main function**

4. ALGORITHM DESCRIPTION

The program contains two overloaded functions with the same name shortestWord. Both functions solve the same task but work with different string representations. The first overload processes a c-string and writes the answer to an output character array. The second overload processes an object of type string and writes the answer to an output object of type string.

For the c-string overload, the algorithm scans the source string from left to right. Spaces are skipped first. After that, the current word beginning is saved, the index moves to the end of the word, and the word length is calculated. If the current word is the first found word or is shorter than the previously saved shortest word, its position and length are stored. After the scan is finished, the shortest word is copied to the result array and terminated with the null character.

For the string overload, the same logic is used, but lengths and positions are applied to the string object. After the shortest word is found, the answer is formed with the method substr.

The main function performs all user interaction. It prints the task number, reads the number of strings, validates that the value is numeric and belongs to the allowed range from 1 to 10, and removes the rest of the first input line from the stream. After that, it fills a two-dimensional character array with user-entered c-strings and creates a second array of objects of type string by copying values from the first array. Then main separately calls the c-string version for every element of the first array and the string version for every element of the second array. The result of each call is printed on the screen. If line input fails, the program prints an error message and terminates.

5. SOURCE CODE

```
#include <iostream>
#include <limits>
#include <string>

void    shortestWord(const char *str, char *result)
{
    size_t  best_start = 0;
    size_t  best_len = 0;
    size_t  i = 0;
    bool    found = false;

    while (str[i])
    {
        while (str[i] == ' ') i++;
        if (!str[i])
            break;
        size_t word_start = i;
        while (str[i] && str[i] != ' ') i++;
        size_t len = i - word_start;
        if (!found || len < best_len)
        {
            best_start = word_start;
            best_len = len;
            found = true;
        }
    }
    if (found)
    {
        for (size_t j = 0; j < best_len; j++)
            result[j] = str[best_start + j];
        result[best_len] = '\0';
    }
    else
        *result = '\0';
}

void    shortestWord(const std::string &str, std::string &result)
{
    size_t  best_start = 0;
    size_t  best_len = 0;
    size_t  len = str.length();
    size_t  i = 0;
    bool    found = false;

    while (i < len)
```

```
    {
        while (i < len && str[i] == ' ') i++;
        if (i >= len)
            break;
        size_t word_start = i;
        while (i < len && str[i] != ' ') i++;
        size_t word_len = i - word_start;
        if (!found || word_len < best_len)
        {
            best_len = word_len;
            best_start = word_start;
            found = true;
        }
    }
    if (found)
        result = str.substr(best_start, best_len);
    else
        result = "";
}

int     main(void)
{
    const int   MAX_STRINGS = 10;
    const int   MAX_LEN = 256;
    char        c_arr[MAX_STRINGS][MAX_LEN];
    std::string s_arr[MAX_STRINGS];
    char        result_c[MAX_LEN];
    std::string result_s;
    int         n;
    int         i;

    std::cout << "Task Nr " << 93642 % 25 << std::endl;
    std::cout << "Enter number of strings: ";
    if (!(std::cin >> n))
    {
        std::cout << "Invalid count" << std::endl;
        return (1);
    }
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

    if (n <= 0 || n > MAX_STRINGS)
    {
        std::cout << "Invalid count" << std::endl;
        return (1);
    }

    i = -1;
```

```
    while (++i < n)
    {
        std::cout << "Enter string " << i + 1 << ": ";
        if (!std::cin.getline(c_arr[i], MAX_LEN))
        {
            std::cout << "Invalid input" << std::endl;
            return (1);
        }
        s_arr[i] = c_arr[i];
    }

    std::cout << std::endl << "Results (c-string version):" << std::endl;
    i = -1;
    while (++i < n)
    {
        shortestWord(c_arr[i], result_c);
        std::cout << "\"" << c_arr[i] << "\" -> \""
            << result_c << "\"" << std::endl;
    }

    std::cout << std::endl << "Results (string version):" << std::endl;
    i = -1;
    while (++i < n)
    {
        shortestWord(s_arr[i], result_s);
        std::cout << "\"" << s_arr[i] << "\" -> \""
            << result_s << "\"" << std::endl;
    }

    return (0);
}
```

6. RUNNING PROGRAM EXAMPLE

```
┌[X]-[16:18-12/04]-[/home/_6r4ndf4th3r/TSI/LAB2_1]
└─_6r4ndf4th3r$make
g++    -c -o main.o main.cpp
g++ -Wall -Wextra -Werror -pedantic main.o -o main
┌[X]-[16:19-12/04]-[/home/_6r4ndf4th3r/TSI/LAB2_1]
└─_6r4ndf4th3r$./main
Task Nr 17
Enter number of strings: 3
Enter string 1: Hello world
Enter string 2: C++ father is Stroustrup
Enter string 3: C father is Ritchie

Results (c-string version):
"Hello world" -> "Hello"
"C++ father is Stroustrup" -> "is"
"C father is Ritchie" -> "C"

Results (string version):
"Hello world" -> "Hello"
"C++ father is Stroustrup" -> "is"
"C father is Ritchie" -> "C"
┌[X]-[16:21-12/04]-[/home/_6r4ndf4th3r/TSI/LAB2_1]
└─_6r4ndf4th3r$
```

Fig. 4. **Running program example**

7. TESTING

Table 1

Testing table

Test Nr.	Input data	Expected result	Actual result	Conclusion
1	n = 1 hello world a test	Shortest word is "a" for both overloads	The program outputs "a" for both overloads	Pass
2	n = 1 hello	Shortest word is "hello" for both overloads	The program outputs "hello" for both overloads	Pass
3	n = 1 abc def ghi	Any 3-letter word may be returned. The implementation returns the first one: "abc"	The program outputs "abc" for both overloads	Pass
4	n = 2 The quick brown fox I am a programmer	For the first string, shortest word is "The". For the second string, shortest word is "I"	The program outputs "The" and "I" for both overloads	Pass
5	n = 1 hello I world	Shortest word is "I" for both overloads	The program outputs "I" for both overloads	Pass
6	n = 1 hello is world	Shortest word is "is" for both overloads	The program outputs "is" for both overloads	Pass
7	n = 1 hello world a	Shortest word is "a" for both overloads	The program outputs "a" for both overloads	Pass
8	n = 1 I love programming	Shortest word is "I" for both overloads	The program outputs "I" for both overloads	Pass

8. CONCLUSIONS

A C/C++ program was developed to identify the shortest word in user-provided strings, implemented using two overloaded functions - one operating on null-terminated c-strings and one on std::string objects. All requirements were fulfilled, including the use of const parameters, prohibition of direct conversions between string types, and keeping all user interaction confined to main.

The implementation iterates over each string character-by-character, tracking the start position and length of the shortest word found without relying on any string-splitting utilities. Both overloads produce identical results, confirming consistent logic across representations. Standard C/C++ library headers (<iostream>, <limits>, <string>) were used without complications. Input validation rejects non-numeric or out-of-range string counts with appropriate error messages.

Limitations include handling of only space-delimited words (no punctuation stripping) and a fixed maximum input length of 255 characters imposed by getline with a static buffer. The program performs reliably for all valid inputs tested. Key challenges: correctly handling edge cases such as single-word strings, strings where all words share the same length, and ensuring the c-string and std::string overloads behave identically. No algorithmic complications were encountered.

Skills acquired include function overloading, manual character-level string traversal, and robust input validation using cin with ignore. The task required approximately 2 hours, primarily spent testing edge cases. This lab reinforced practical understanding of function overloading, string representations in C++, and the differences between c-string and std::string manipulation.